
STACK SÊNIOR

Feche o ciclo do PR no GitHub

Review Cycle: o complemento do boilerplate para a IA revisar, responder e fechar **o seu PR**.

post-review • do-review • check-review

Guia de instalação e uso



Sumário

1	Leia antes de instalar	1
2	O que é o Review Cycle	3
3	Instalação	5
4	O ciclo em quatro etapas	6
5	Contratos de API	9
6	Quando algo dá errado.	11

Guia curto de propósito. Leia o aviso da primeira página antes de qualquer coisa. Depois, instalação e o ciclo em quatro etapas cabem em uma sentada.

PARTE 1

Leia antes de instalar

**SE VOCÊ NÃO TEM
O BOILERPLATE COMPLETO,
ESTE PRODUTO PODE NÃO
FUNCIONAR DA MESMA FORMA**

O Review Cycle é um **complemento** do Stack Sênior. As três skills deste pacote leem arquivos, regras e hooks que só existem em um projeto criado a partir do boilerplate. Em outro repositório elas até rodam, mas as referências não batem e o review perde a régua.

1.1 O que este pacote espera encontrar no seu projeto

Dependências do boilerplate

Arquivo ou recurso	Para que o ciclo usa
.claude/docs/guardrails-catalog.md	Catálogo A-G: a régua de julgamento das lentes. O <code>install.sh</code> recusa instalar sem ele.
CLAUDE.md , seção <i>Branch base</i>	Base do PR. Sem ela, o review compara contra a branch errada.
CLAUDE.md , seção <i>Contrato de API</i>	<code>\$API_CONTRACT_PATH</code> , que ativa a lente de contratos (opcional).
VERCEL-OXLINT-MAPPING.md	Rulebook das lentes de performance.
MEMORY.md (ADRs)	Âncoras para contestar ou sustentar um finding; ADR-021 define o gate de review.
tooling/agent-hooks/	Os hooks que o review considera gate; editar um para o PR passar é finding <i>blocker</i> .
Regras boilerplate/* no oxlint	O que já é gate mecânico e o review não repete.
.agents/skills/pre-review	Etapa 1 do ciclo. Vem na base, não neste pacote.

CUIDADO | Sem esses arquivos

As skills citam regras que não existem, a lente 8 nunca roda, o selo da pré-review não é encontrado e a conclusão do review sai sem fundamento. Não é um erro que aparece na tela: é um review com aparência de completo e conteúdo vazio. Instale sobre o boilerplate.

1.2 Ferramentas necessárias

- gh autenticado com permissão de escrita no repositório do PR (`gh auth status`);
- jq e git;
- um dos hosts: Claude Code, Codex ou Cursor.

PARTE 2

O que é o Review Cycle

O QUE VOCÊ LEVA DESTE CAPÍTULO

O boilerplate cuida do caminho até o `gh pr create`. Este pacote cuida do que vem depois: quem revisa o PR, quem responde aos achados e quem fecha o ciclo. Três skills, cada uma com um papel fixo.

A base entrega a `pre-review`: o julgamento local antes de abrir o PR, com o selo que libera o `gh pr create`. Depois disso, o PR fica no GitHub esperando alguém. O Review Cycle coloca o agente nesse lugar, com a mesma régua da `pre-review` e separação clara entre quem revisa e quem responde.

O ciclo completo

Etapa	Skill	Quem	O que sai
1	<code>pre-review (base)</code>	Autor, antes do PR	Relatório local + selo <code>.agents/tmp/pre-review-<HEAD>.ok</code>
2	<code>post-review</code>	Reviewer, PR aberto	Review adversarial publicado no GitHub: inline + comentário geral
3	<code>do-review</code>	Autor	Triagem com você; fix do que procede, contestação com fonte do que não procede; resposta em toda thread
4	<code>check-review</code>	Reviewer, sessão nova	Re-julga cada resposta pelo código, resolve threads, aprova só com tudo fechado

REGRA | Reviewer e autor em sessões separadas

`post-review` e `check-review` são o reviewer; `do-review` é o autor. Rodar o `check-review` na mesma sessão que escreveu as respostas anula a separação que o ciclo existe para dar. A skill detecta e registra quando isso acontece.

2.1 Como as skills estão organizadas

O conteúdo canônico mora em `.agents/skills/<skill>/SKILL.md`. Cada host tem um wrapper fino em `.claude/skills`, `.codex/skills` e `.cursor/skills` que só aponta para o canônico e declara o que é específico daquele host. Mudou a regra? Muda no canônico e os três herdam.

Por host

Host	Como as lentes do post-review rodam
Claude Code	Tool Workflow (ultracode): lentes 1 a 8 em paralelo, lente 9 como estágio seguinte. Modelo Opus nos agentes.
Codex	Subagentes do Codex, modelo e reasoning herdados da sessão.
Cursor	Subagentes via Task, modelo herdado da sessão.

PARTE 3

Instalação

O QUE VOCÊ LEVA DESTE CAPÍTULO

Um comando copia as três skills para o seu projeto. Nada de settings, hooks ou outras skills é tocado.

Instalar sobre um projeto Stack Sênior

```
./install.sh /caminho/do/seu/projeto
```

O script confere se o alvo tem [.claude/docs/guardrails-catalog.md](#). Sem ele, recusa com a mensagem “Alvo não parece um projeto Stack Sênior”. Esse é o aviso da primeira página em forma de código.

O que é copiado, por skill, para [.agents](#), [.claude](#), [.codex](#) e [.cursor](#):

- [skills/post-review/](#) (SKILL.md, lenses.md, project-catalog.md);
- [skills/do-review/SKILL.md](#);
- [skills/check-review/SKILL.md](#).

Rodar o `install.sh` de novo substitui essas pastas pela versão do pacote. Personalizações suas dentro delas são perdidas; faça-as em um fork do canônico se precisar.

PRÓXIMO PASSO: confira no host

Abra o projeto no Claude Code, Codex ou Cursor e digite `/post-review`. A skill deve aparecer na lista. Se não aparecer no Codex, copie também para `$CODEX_HOME/skills/<skill>/`.

3.1 Opcional: ativar a lente de contratos

Na seção *Contrato de API* do [CLAUDE.md](#), defina `$API_CONTRACT_PATH`. Com ela, a `post-review` compara o diff contra a spec OpenAPI do seu backend. Sem ela, a lente 8 é pulada e o review registra a ressalva. Detalhes no capítulo **Contratos de API**.

PARTE 4

O ciclo em quatro etapas

O QUE VOCÊ LEVA DESTE CAPÍTULO

O que cada skill faz, o que ela se recusa a fazer e o que você precisa decidir em cada ponto.

4.1 Etapa 1: pre-review (vem na base)

Roda antes do `gh pr create`, grava o selo do HEAD atual e libera o hook `pre-pr-review-gate.ts`. Está documentada no guia do boilerplate. O `post-review` lê o selo: se existir para o mesmo SHA, ele foca em integração e callers fora do diff em vez de reabrir o catálogo A-G inteiro.

4.2 Etapa 2: post-review

Reviewer, com o PR aberto

```
/post-review 42      # vazio = PR da branch atual
```

O que acontece:

1. pina o SHA do head do PR e usa esse SHA em tudo (diff, lentes, publicação);
2. lê [AGENTS.md](#), [ARCHITECTURE.md](#), [CODE-PATTERN.md](#) e [MEMORY.md](#);
3. se o diff toca borda de API e há contrato definido, baixa a spec para a lente 8;
4. roda as lentes 1 a 8 em subagentes separados, cada uma com uma única preocupação;
5. a lente 9 (devil's advocate) tenta refutar cada finding e deduplica contra o que já está no PR;
6. o contexto principal relê cada sobrevivente no código real e classifica: *blocker*, *high*, *medium*, *low*;
7. publica um review com comentários inline (cinco seções obrigatórias: evidência, o que pode causar, por que resolver, recomendação, referência) e um comentário geral com a conclusão.

As lentes

#	Foco
1	Arquitetura e simplicidade: over-engineering, camadas sem justificativa, regra de negócio no front
2	React e frontend: componentes aninhados, effects evitáveis, lógica pesada em <code>.tsx</code>
3	Fluxos, erros e fallbacks: <code>catch</code> que engole, <code>?? 0</code> em shape garantida, dual-path
4	Legibilidade e manutenção: duplicação, nomes genéricos, campo sem leitor
5–7	Performance Vercel: waterfalls e server, bundle e client, re-render e rendering
8	Contratos de API: diff contra a spec em <code>\$API_CONTRACT_PATH</code> (condicional)
9	Devil's advocate: refuta, funde duplicatas entre lentes, marca duplicata contra comentário já publicado

REGRA | O que o post-review nunca faz

Não altera código, não commita, não faz push e nunca aprova. O `event` do review é sempre `COMMENT`. Se o head do PR mudar durante o review, a publicação é abortada e o ciclo reinicia no SHA novo.

A conclusão no comentário geral é uma de três: *Approve*, *Approve with non-blocking suggestions* ou *Request changes*. Só sai da família *Approve* quando o review foi completo: lente 8 executada se havia borda de API, e nenhum arquivo do diff ficou sem leitura por nenhuma lente.

4.3 Etapa 3: do-review

Autor, depois do post-review

```
/do-review 42
```

Roda no contexto principal porque a conversa com você faz parte do fluxo:

1. coleta toda thread aberta e todo finding do comentário geral; falha de consulta interrompe a skill, nunca vira “nenhum finding”;
2. para cada item, forma o próprio juízo e recomenda: *acatar*, *contestar* ou *discutir*;
3. você decide, em lote para os consensuais e um a um para os ambíguos;
4. só depois de todo item decidido, aplica os fixes (hard cutover, sem afrouxar gate) e escreve as contestações com âncora verificável;

5. commit por domínio, push com sua confirmação;
6. responde cada thread: “corrigido em <sha>” ou a justificativa.

CUIDADO | Contestação sem fonte não vai para o PR

“É intencional” ou “sempre foi assim” vão ser rejeitados no check-review. Uma contestação cita ADR em [MEMORY.md](#), o contrato em `$API_CONTRACT_PATH`, uma discussão no PR ou mostra a leitura errada do reviewer com arquivo e linha. Sem isso, o item volta para a triagem.

O do-review nunca resolve thread nem aprova: isso é do reviewer.

4.4 Etapa 4: check-review

Reviewer, em sessão nova

```
/check-review 42
```

1. monta o inventário: threads do review do agent mais os findings do comentário geral, e confere que a soma bate com a contagem publicada no post-review (inventário vazio ou furado interrompe);
2. julga cada item pelo estado do código no head atual: *corrigido*, *justificado sem mudança* ou *pendente*;
3. detecta código novo além dos fixes e o marca como pendência que pede um post-review novo;
4. apresenta o placar a você antes de publicar qualquer coisa;
5. responde cada thread, resolve só depois de a resposta ser aceita pela API, e aprova apenas com tudo fechado e com a sua confirmação.

Threads de revisores humanos ficam intactas, mas abertas bloqueiam a aprovação.

PRÓXIMO PASSO: primeiro ciclo completo

Escolha um PR pequeno que já passou pela pré-review. Rode `/post-review` numa sessão, `/do-review` em outra, e `/check-review` numa terceira. Leia o relatório de cada uma no terminal: ele diz o que foi publicado, o que foi descartado e por quê.

PARTE 5

Contratos de API

O QUE VOCÊ LEVA DESTE CAPÍTULO

A lente 8 compara o que o PR consome do backend com o que a spec documenta. Ela só roda quando você diz onde a spec está.

Na seção *Contrato de API* do [CLAUDE.md](#), defina `$API_CONTRACT_PATH` com um dos dois formatos:

Formatos aceitos

Formato	Exemplo
Path local	<code>docs/openapi.yaml</code> ou uma pasta com vários YAMLS
Repositório GitHub	<code>owner/repo/swagger@main</code> (lido via <code>gh api</code>)

Borda de API, para o review, é `*.request.ts`, `*.server.ts`, `endpoints.ts` e qualquer `*.schema.ts` consumido por esses módulos. Schema de formulário local não conta.

O que acontece em cada situação

Situação	Resultado
Diff sem borda de API	Lente 8 pulada; registrado no relatório.
Borda de API + spec acessível	Lente 8 roda. Divergência que quebra request ou parse em runtime é <i>blocker</i> ; as demais, <i>high</i> .
Endpoint sem doc na spec	Lente roda com o que tem; o endpoint vira nota “consumido sem documentação”.
Spec inacessível ou variável não definida	Lente pulada com ressalva no comentário geral. A conclusão não pode ser <i>Approve</i> enquanto houver borda de API sem contrato verificado.

POR QUE | A lente não re-infere contrato do próprio código

Comparar o PR com ele mesmo sempre passa. A régua é a spec: campo validado que a doc não tem, narrowing defensivo para shape que o contrato não produz e re-derivação de campo que a API já tipou são findings porque a doc diz que não deveriam existir. Se a

suspeita é de doc desatualizada, o finding vira recomendação de sync na spec; o PR não é absolvido por isso.

PARTE 6

Quando algo dá errado

Sintoma e saída

Sintoma	O que fazer
<code>install.sh</code> recusa: “Alvo não parece um projeto Stack Sênior”	O alvo não tem .claude/docs/guardrails-catalog.md . Instale o boilerplate primeiro. Ver a primeira página.
“head drifted” no post-review ou check-review	O PR recebeu commit durante o review. Rode a skill de novo; ela reinicia no SHA novo.
Lente 8 pulada	Falta <code>\$API_CONTRACT_PATH</code> no CLAUDE.md ou a spec está inacessível. O review segue, sem aprovar limpo.
<code>gh api</code> retorna 403 ou 404	<code>gh auth status</code> ; confirme permissão de escrita em PR e que o repositório é o do PR (<code>gh repo view</code>).
Check-review interrompe com “inventário vazio ou divergente”	A contagem de threads não bate com o que o post-review publicou. Alguém resolveu threads por fora ou o review do agent não foi encontrado pela atribuição. Não force: confira o PR no GitHub.
Skill não aparece no Codex	Copie também para <code>\$CODEX_HOME/skills/<skill>/. </code>
Review citando regra avante/* ou Jira	Versão antiga do pacote. Rode <code>./check.sh</code> no pacote; ele acusa resíduo de outro projeto. Reinstale.
Autor e reviewer na mesma sessão	O check-review registra isso no relatório e no fechamento. Abra uma sessão nova para o reviewer.

POR QUE | Por que o ciclo interrompe em vez de seguir

Falha de consulta virando “nenhum finding” ou inventário vazio virando “tudo aceito” é o pior resultado possível: um PR aprovado sem que ninguém tenha olhado. Cada skill prefere parar com relatório a seguir com dado incompleto.

6.1 Verificar o pacote

Autoteste

```
./check.sh
```

Confere que todo wrapper aponta para um canônico existente e que não há resíduo de outro projeto nas skills. Saída esperada: OK.